

TECNOLOGÍAS DE LA INFORMACIÓN EN LÍNEA

DESARROLLO DE APLICACIONES WEB

3 créditos

**Profesor:**

Ing. Manuel Bravo, Mg.

Ing. Tatiana Cobeña Macías, Mg.

Ing. Edison Solorzano, MG.

Asignatura	Semestre
DESARROLLO DE APLICACIONES WEB	Sexto

Tutorías: El profesor asignado, se publicará en el entorno virtual de aprendizaje online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección **CAFETERÍA VIRTUAL**.

PERÍODO ACADÉMICO:

MAYO 2023 / SEPTIEMBRE 2023

Contenido

Unidad I: Programación web en el cliente	2
Tema 1: Introducción a la Web	2
Tema 2: HTML	5
Tema 3: CSS	14
Tema 4: JAVASCRIPT	20



Resultado de aprendizaje de la asignatura

Este curso provee a los estudiantes el conocimiento y la experiencia práctica para diseñar e implementar aplicaciones web cumpliendo con los estándares actuales y las buenas prácticas de programación que faciliten su mantenibilidad, escalabilidad y adaptabilidad.



Ilustraciones gráficas



Sabías que. - La presente imagen dentro del manual mostrara información interesante y novedosa de la asignatura.



Recuerde que. - La presente imagen dentro del manual permite recordar información que es relevante y que vas necesitar en tu vida profesional.



Comprueba tu aprendizaje. - Es un cuestionario de un conjunto de preguntas que se confecciona para obtener información con algún objetivo en concreto. Por cada tema de la unidad se tendrá un cuestionario que el estudiante debe resolver entre preguntas teóricas y prácticas.



Videos. - Para complementar contenidos de la unidad dentro del manual se tiene videos que permitirán al estudiante revisar y explorar conocimientos auditivos y visuales.



Curiosidades. - La presente imagen en el manual mostrara información que debes conocer de la asignatura.



Datos útiles. - La presente imagen en el manual mostrara información que deberás tomar en cuenta en otras unidades de la asignatura o de otras asignaturas en semestres superiores.



DESARROLLO DE APLICACIONES WEB



Unidad I: Programación web en el cliente

Resultado de aprendizaje de la unidad: Construir aplicaciones web que separen convenientemente datos, código y presentación, y que utilicen adecuadamente la arquitectura cliente-servidor.

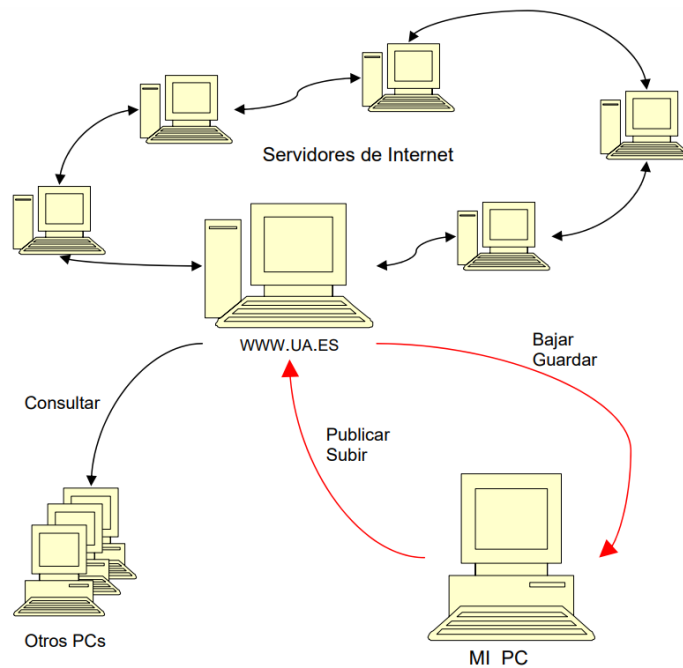


Tema 1: Introducción a la Web

Internet es una red de ordenadores conectados en todo el mundo que ofrece diversos servicios a sus usuarios, como pueden ser el correo electrónico, el chat o la web. Todos los servicios que ofrece Internet son llevados a cabo por miles de ordenadores que están permanentemente encendidos y conectados a la red, esperando que los usuarios les soliciten los servicios y sirviéndolos una vez son solicitados.

Estos ordenadores son los servidores, los hay que ofrecen correo electrónico, otros hacen posible nuestras conversaciones por chat, otros la transferencia de ficheros o la visita a las páginas web y así hasta completar la lista de servicios de Internet.

Cuando copiamos una página en el servidor para que sea accesible por todo el mundo hablamos de subir o publicar (upload). Y cuando nos guardamos algo de internet en nuestro ordenador lo llamamos bajar o guardar (download).



Por otra parte, para López (2022), la World Wide Web, o simplemente red mundial, es el universo de información accesible a través de Internet, una fuente inagotable del conocimiento humano. La web se puede generalizar en:

- Aplicaciones web.
- Páginas Web.
- Buscadores web.
- Servidores web.

¿Qué son las aplicaciones web?

Las aplicaciones web son aquellas aplicaciones informáticas que están alojadas en un servidor web (Internet) y que pueden ser accedidas mediante un navegador.

¿Qué son las páginas web?

Es un documento o información electrónica capaz de contener texto, sonido, vídeo, programas, enlaces, imágenes, hipervínculos, entre otras cosas.



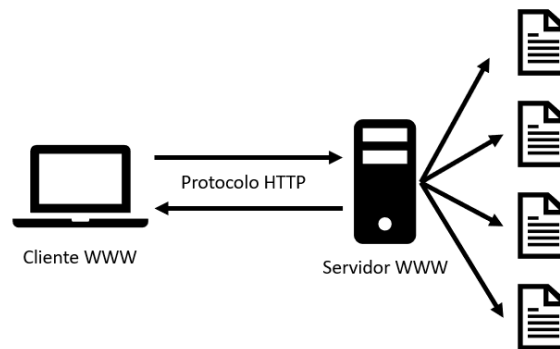
Sabías que. - De manera general, las páginas web son más informativas y muy creativas. Para un rápido desarrollo, se recomienda usar: HTML y CSS.

¿Qué son los buscadores web?

Un buscador web o motor de búsqueda es un sistema informático que busca archivos almacenados en servidores web gracias a una especie de red compuesta por millones de hilos comunicados que permiten que el sistema detecte palabras referentes a su contenido.

¿Qué son los servidores web?

Un servidor web o servidor HTTP es un software, espacio o dispositivo virtual que le brinda almacenamiento y estructura a las aplicaciones o sitios web para que puedan almacenar, procesar y desplegar sus datos.

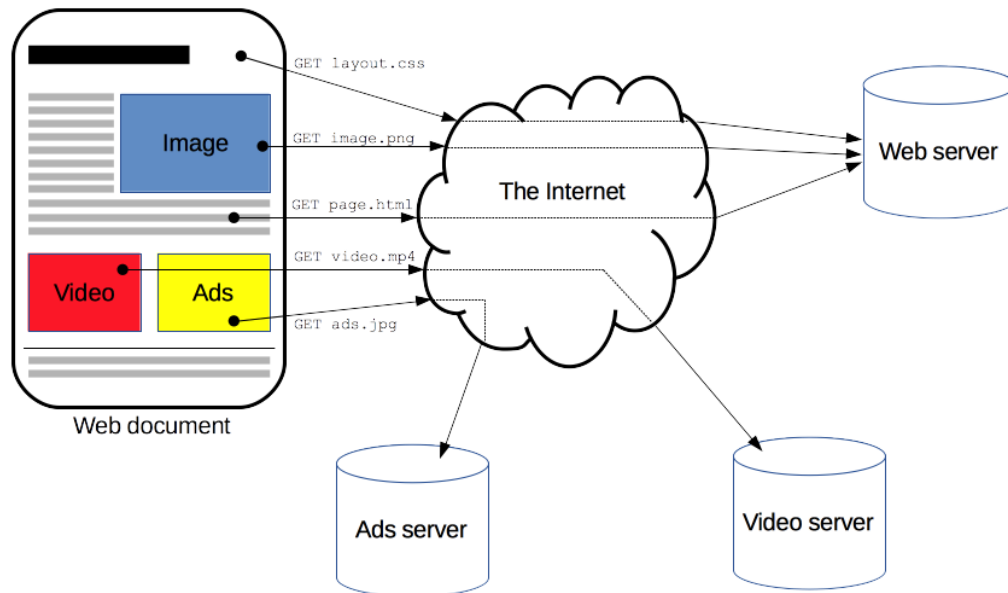


¿Qué es HTTP?

HTTP o Hypertext Transfer Protocol, es un protocolo web que permite realizar una petición de datos y recursos, en los diversos documentos de la web, por ejemplo: HTML.

HTTP, es la base de cualquier intercambio de datos en la web y su estructura es cliente-servidor.





Importancia del Desarrollo Web

El desarrollo web cobra una relevancia especial en los tiempos actuales, donde prácticamente la inmensa población a nivel mundial está conectada a Internet y la utiliza a diario como herramienta de trabajo o consulta. Y es que, realmente resulta difícil de comprender que aún existan personas que no utilicen la web como un medio de comunicación e investigación.

Ventajas de ser, desarrollador web

- ✓ Trabajo remoto.
- ✓ Reutilización de código.
- ✓ Mayor oferta laboral.
- ✓ Puedes trabajar de manera independiente.
- ✓ Puede vender su trabajo de forma online.
- ✓ Contar con muchas herramientas o frameworks que facilitan la tarea.



Tema 2: HTML

HyperText Markup Language (Lenguaje de marcado de hipertexto) o HTML es un lenguaje de marcado predominante para la elaboración de páginas web que se utiliza para describir y traducir la estructura y la información en forma de texto, así como para complementar el texto con objetos tales como imágenes.

El HTML se escribe en forma de etiquetas o TAGs, contenidas dentro de corchetes angulares(< y >).

```
<html>
  <head>
    <title>Ejemplo</title>
  </head>
  <body>
    <p>Ejemplo</p>
  </body>
</html>
```

CARACTERES ESPECIALES DE HTML

Los caracteres especiales se deben convertir en HTML para mostrarse correctamente en un navegador.

Carácter	Código HTML	Carácter	Código HTML
á	á	Á	Á
é	é	É	É
í	í	Í	Í
ó	ó	Ó	Ó
ú	ú	Ú	Ú
ü	ü	Ü	Ü
ñ	ñ	Ñ	Ñ
¡	¡	¿	¿

DOCUMENTO HTML

```
plantilla_basica.html x
introduccion > plantilla_basica.html > ...
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="UTF-8">
5    <meta http-equiv="X-UA-Compatible" content="IE=edge">
6    <meta name="viewport" content="width=device-width, initial-scale=1.0">
7    <title>Mi primer plantilla de HTML</title>
8  </head>
9  <body>
10   <h1>Hola Mundo</h1> <!--etiqueta h1 para titulos-->
11   <p>Estoy aprendiendo desde cero HTML</p> <!--etiqueta párrafo-->
12 </body>
13 </html>
```

EXPLICACIÓN DE CÓDIGO (ETIQUETADO)

- La declaración `<!DOCTYPE html>` define que este documento es un documento HTML5.
- El elemento `<html>` es el elemento raíz de una página HTML
- El elemento `<head>` contiene meta información sobre la página HTML
- El elemento `<title>` especifica un título para la página HTML (que se muestra en la barra de título del navegador o en la pestaña de la página)
- El elemento `<body>` define el cuerpo del documento, y es un contenedor para todos los contenidos visibles, como títulos, párrafos, imágenes, hipervínculos, tablas, listas, etc.
- El elemento `<h1>` define un título grande.
- El elemento `<p>` define un párrafo.

¿QUÉ ES UN ELEMENTO HTML?

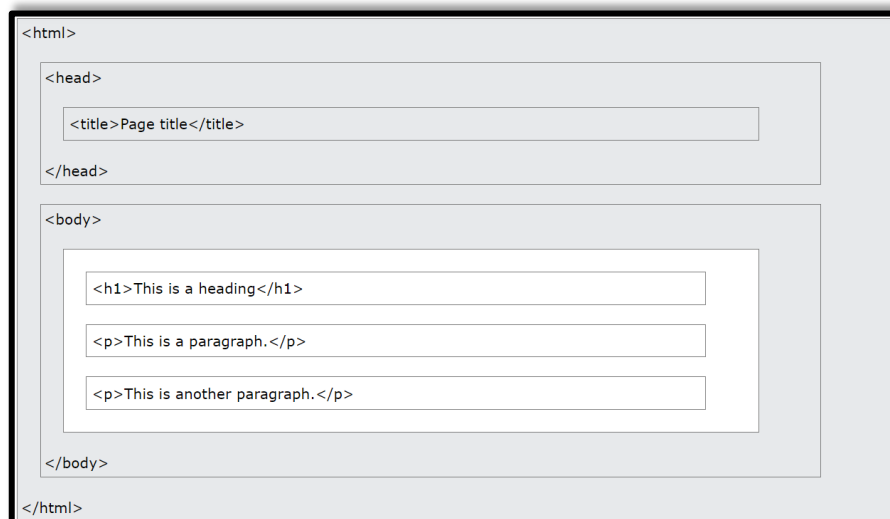
Un elemento HTML está definido por una etiqueta inicial, un contenido y una etiqueta final: `<tagname> El contenido va aquí... </tagname>`

El elemento HTML es todo lo que va desde la etiqueta de inicio hasta la etiqueta de fin:

`<h1>Mi primer encabezado</h1>`

`<p>Mi primer párrafo</p>`

ESTRUCTURA DE LA PÁGINA HTML



HISTORIA DEL HTML

Desde los primeros días de la World Wide Web, ha habido muchas versiones de HTML:

Año	Versión
1989	Tim Berners-Lee inventó www
1991	Tim Berners-Lee inventó HTML
1993	Dave Raggett redactó HTML+
1995	El Grupo de Trabajo de HTML definió HTML 2.0
1997	Recomendación del W3C: HTML 3.2
1999	Recomendación del W3C: HTML 4.01
2000	Recomendación del W3C: XHTML 1.0
2008	Primer borrador público del WHATWG HTML5
2012	WHATWG HTML5 Living Standard
2014	Recomendación del W3C: HTML5
2016	Recomendación candidata del W3C: HTML 5.1
2017	Recomendación del W3C: HTML5.1 2ª edición
2017	Recomendación del W3C: HTML5.2

RECOMENDACIONES SOBRE EDITORES HTML

Un simple editor de texto es todo lo que necesitas para aprender HTML.

- Es recomendable aprender HTML con Notepad o TextEdit
- Las páginas web pueden crearse y modificarse utilizando editores profesionales de HTML.
- Sin embargo, para aprender HTML recomendamos un simple editor de texto como Notepad (Windows) o TextEdit en (Mac).
- Creemos que el uso de un simple editor de texto es una buena manera de aprender HTML.
- Sigue los siguientes pasos para crear tu primera página web con Notepad o TextEdit.

EDITORES

- Notepad++
- Visual Studio Code
- Sublime Text



- Komodo Edit
- Bloc de notas
- Atom
- Dreamweaver



ATRIBUTOS DE HTML

- Los atributos HTML proporcionan información adicional sobre los elementos HTML.

Características

- Todos los elementos HTML pueden tener atributos
- Los atributos siempre se especifican en la etiqueta inicial
- Los atributos suelen venir en pares nombre/valor como: name="value".

Atributos de HTML

- **El atributo href**

La etiqueta <a> define un hipervínculo. El atributo href especifica la URL de la página a la que va el enlace:

Ejemplo:

```
<a href="url">link text</a>
```

```
<a href="www.utm.edu.ec">Visita la UTM</a>
```

Por defecto, los enlaces aparecerán de la siguiente manera en todos los navegadores:

- ✓ Los enlaces no visitados aparecen subrayados y en color azul.
- ✓ Los enlaces visitados aparecen subrayados y en color púrpura.

- **Enlaces HTML - El atributo target**

Por defecto, la página enlazada se mostrará en la ventana actual del navegador. Para cambiar esto, debe especificar otro destino para el enlace. El atributo target especifica dónde abrir el documento enlazado.

El atributo target puede tener uno de los siguientes valores:

- ⊕ **_self** - Por defecto. Abre el documento en la misma ventana/pestaña en la que se hizo clic.
- ⊕ **_blank** - Abre el documento en una nueva ventana o pestaña.
- ⊕ **_parent** - Abre el documento en el marco principal.
- ⊕ **_top** - Abre el documento en el cuerpo completo de la ventana.

Ejemplo:

```
<a href="url" target="...">link text</a>
```

```
<a href="www.utm.edu.ec" target="_blank">Visita la UTM</a>
```

- **Enlace a una dirección de correo electrónico**

Utilice mailto: dentro del atributo href para crear un enlace que abra el programa de correo electrónico del usuario (para permitirle enviar un nuevo correo):

Ejemplo:

```
<a href="mailto:someone@example.com">Found email</a>
```

```
<a href="mailto:oaigs@outlook.es" target="_blank">Direccionar a correo</a>
```

- **El atributo src**

La etiqueta se utiliza para incrustar una imagen en una página HTML. El atributo src especifica la ruta de la imagen que se va a mostrar:

Ejemplo:

```

```

```

```

Nota: La etiqueta debe contener también los atributos width y height, que especifican la anchura y la altura de la imagen (en píxeles).

- **El atributo alt**

El atributo alt requerido para la etiqueta especifica un texto alternativo para una imagen, si la imagen por alguna razón no puede ser mostrada. Esto puede deberse a una conexión lenta, a un error en el atributo src o a que el usuario utilice un lector de pantalla.

Ejemplo:

```

```

```

```

▪ El atributo style

El atributo style especifica un estilo en línea para un elemento. El atributo style anulará cualquier estilo establecido globalmente, por ejemplo, los estilos especificados en la etiqueta <style> o en una hoja de estilo externa.

Ejemplo:

```
<h1 style="propiedades_css">This is a header</h1>
```

```
<h1 style="color:blue;text-align:center">This is a header</h1>
```

Si desea indagar más en atributos de HTML, puede revisar el siguiente link:

 https://www.w3schools.com/tags/ref_attributes.asp

REGLAS DE HTML

Las etiquetas se cierran en el mismo orden en que se abren:

```
<html><body>Texto</html></body>
<html><body>Texto</body></html>
```

Los nombres de las etiquetas y sus atributos se escriben preferentemente en minúsculas:

```
<HTML><BODY>Texto</BODY></HTML>
<html><body>Texto</body></html>
```

Los atributos van entre comillas:

```
<p align=left>Texto<p>
<p align="left">Texto<p>
```

Los valores de los atributos no se contraen o comprimen:

```
<p left>Texto<p>
<p align="left">Texto<p>
```

Toda etiqueta debe cerrarse:

```
<html><body>Texto</body>
<html><body>Texto</body></html>
```

Etiquetas de línea y salto de línea:

`<hr>` `<br>`

FORMULARIOS

Los formularios sirven para poder enviar información desde el browser hacia algún otro lugar (generalmente hacia el servidor mismo).

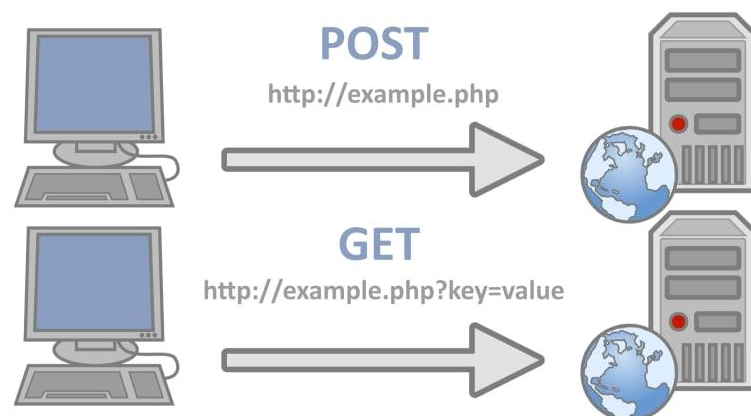
- Los formularios pueden enviar la información de dos formas distintas: POST y GET
- Cada item (textarea, text) de un formulario tiene que tener un nombre definido (name).

Input ítems

Sintaxis básica: `<input type="item" name="nombre" value="valor">`

- Text
- Password
- Submit
- Checkbox
- Radio
- Select

MÉTODOS DE ENVIO DE DATOS



Cuando se envía información usando el método POST, esta se envía de forma “escondida” utilizando el browser. Se usa generalmente para los formularios de tipo Login y Password.

Cuando se envía información usando el método GET, esta se envía de forma “explícita” en la URL (dirección). Se usa generalmente para crear links dinámicos.

Ejercicio usando HTML y CSS (Formulario).

DATOS PARA EL REGISTRO EN PLATAFORMA

Oscar

López

oalgs@outlook.es

Ciudad de origen: Chone

Color favorito:

Todo lo puedo en Cristo que me fortalece.

Registrar
Borrar form

```

<form action="registro.php" method="get">
  <div class="registro_usuario">
    <!--Aquí es donde usted debe comenzar a utilizar las etiquetas de HTML para su formulario-->
    <label for="datos"><h4 class="text_mayus">Datos para el registro en plataforma</h4></label>
    <input type="text" class="label" name="name_user" placeholder="Ingrese sus nombres"><br>
    <input type="text" class="label" name="ape_user" placeholder="Ingrese sus apellidos"><br>
    <input type="text" class="label" name="email_user" placeholder="Ingrese su correo"><br>
    <label for="ciudad" style="color: ■ white;">Ciudad de origen: </label><span>
    <select name="ciudad">
      <option value="Portoviejo">Portoviejo</option>
      <option value="Manta">Manta</option>
      <option value="Chone">Chone</option>
      <option value="Santa Ana">Santa Ana</option>
      <option value="Paján">Paján</option></span>
    </select>
    <br>
    <br>
    <label for="color_favorito" style="color: ■ white;">Color favorito: </label>
    <input type="color" name="color_fav" placeholder="Elija su color favorito"><br><br>
    <textarea name="textarea_frase" id="" cols="30" rows="6" placeholder="Aquí podrá colocar su frase célebre favorita"></textarea><br><br>
    <div class="botoncito_registrar">
      <input type="submit" name="insertar" style="background-color: ■ darkgreen; color: ■ white;" value="Registrar">
      <input type="reset" style="background-color: ■ brown; color: ■ white;" name="borrar" value="Borrar form">
    </div>
  </div>
</form>
          
```



Recuerde que. – HTML, sirve para construir la estructura o esqueleto de su aplicación web.

```

FORMULARIOS > styles > styles.css > ...
1  .registro_usuario {
2      padding: 20px;
3      text-align: center;
4      background: ■ #DE6262;
5      /* fallback for old browsers */
6      background: -webkit-linear-gradient(to right, ■ #FFB88C, ■ #DE6262);
7      /* Chrome 10-25, Safari 5.1-6 */
8      background: linear-gradient(to right, ■ #FFB88C, ■ #DE6262);
9      /* W3C, IE 10+/ Edge, Firefox 16+, Chrome 26+, Opera 12+, Safari 7+ */
10 }
11
12 .label {
13     padding: 20px;
14     margin: 10px;
15     border-radius: 20px;
16     border: 2px ■ black solid;
17 }
18
19 .text_mayus {
20     text-transform: uppercase;
21     color: ■ white;
22     text-align: center;
23 }
24
25 .botoncito_registrar {
26     display: flex;
27     flex-direction: row;
28     justify-content: center;
29 }
    
```



Recuerde que. – CSS, sirve para darle estilo a su documento HTML. (Colores, efectos, etc.)

En la sesión de clases, se realizarán pruebas de envío de información, mediante método POST y GET.



Tema 3: CSS

CSS es un lenguaje de hojas de estilos creado para controlar el aspecto o presentación de los documentos electrónicos definidos con HTML y XHTML.

- CSS es la mejor forma de separar los contenidos y su presentación y es imprescindible para crear páginas web complejas.
- Separar la definición de los contenidos y la definición de su aspecto presenta numerosas ventajas, ya que obliga a crear documentos HTML/XHTML bien definidos y con significado completo (también llamados "documentos semánticos").

SOPORTE DE CSS EN LOS NAVEGADORES

Navegador	Motor	CSS 1	CSS 2.1	CSS 3
Internet Explorer	Trident	Completo desde la versión 6.0	Casi completo desde la versión 7.0	Prácticamente nulo
Firefox	Gecko	Completo	Casi completo	Selectores, pseudo-clases y algunas propiedades
Safari	WebKit	Completo	Casi completo	Todos los selectores, pseudo-clases y muchas propiedades
Opera	Presto	Completo	Casi completo	Todos los selectores, pseudo-clases y muchas propiedades
Google Chrome	WebKit	Completo	Casi completo	Todos los selectores, pseudo-clases y muchas propiedades

FUNCIONAMIENTO BÁSICO DE CSS

Antes de la adopción de CSS, los diseñadores de páginas web debían definir el aspecto de cada elemento dentro de las etiquetas HTML de la página. El siguiente ejemplo muestra una página HTML con estilos definidos sin utilizar CSS:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Ejemplo de estilos sin CSS</title>
</head>

<body>

<h1><font color="red" face="Arial" size="5">Titular de la página</font></h1>

<p><font color="gray" face="Verdana" size="2">Un párrafo de texto no muy
```


INCLUIR CSS EN UN DOCUMENTO XHTML

Una de las principales características de CSS es su flexibilidad y las diferentes opciones que ofrece para realizar una misma tarea.

- De hecho, existen tres opciones para incluir CSS en un documento HTML.

Incluir CSS en el
mismo
documento HTML

Definir CSS en un
archivo externo

Incluir CSS en los
elementos HTML

INCLUIR CSS EN EL MISMO DOCUMENTO HTML

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Ejemplo de estilos CSS en el propio documento</title>
<style type="text/css">
  p { color: black; font-family: Verdana; }
</style>
</head>

<body>
<p>Un párrafo de texto.</p>
</body>
</html>
```

DEFINIR CSS EN UN ARCHIVO EXTERNO

En el siguiente ejemplo, se crea un archivo de texto, se cambia su nombre a estilos.css y se incluye el siguiente contenido:

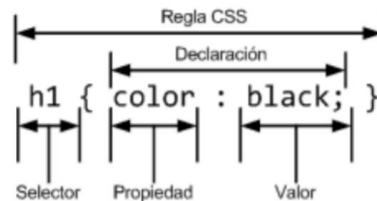
```
p { color: black; font-family: Verdana; }
```

A continuación, en la página HTML se utiliza la etiqueta para enlazar el archivo CSS externo que tiene los estilos que va a utilizar la página:

```
<link rel="stylesheet" type="text/css" href="/css/estilos.css" media="screen" />
```


COMPONENTES DEL ESTILO CSS BÁSICO

CSS define una serie de términos que permiten describir cada una de las partes que componen los estilos CSS. El siguiente esquema muestra las partes que forman un estilo CSS muy básico:



Los diferentes términos se definen a continuación:

- **Regla:** cada uno de los estilos que componen una hoja de estilos CSS.

Cada regla está compuesta de una parte de "selectores", un símbolo de "llave de apertura" ({), otra parte denominada "declaraciones" y, por último, un símbolo de "llave de cierre" (}).

- **Selector:** indica el elemento o elementos HTML a los que se aplica la regla CSS.
- **Declaración:** especifica los estilos que se aplican a los elementos.

Está compuesta por una o más propiedades CSS.

- **Propiedad:** permite modificar el aspecto de una característica del elemento.
- **Valor:** indica el nuevo valor de la característica modificada en el elemento.

MEDIOS CSS

Una de las características más importantes de las hojas de estilos CSS es que permiten definir diferentes estilos para diferentes medios o dispositivos: pantallas, impresoras, móviles, proyectores, etc.

Medio	Descripción
all	Todos los medios definidos
braille	Dispositivos táctiles que emplean el sistema braille
embosed	Impresoras braille
handheld	Dispositivos de mano: móviles, PDA, etc.
print	Impresoras y navegadores en el modo "Vista Previa para Imprimir"
projection	Proyectores y dispositivos para presentaciones
screen	Pantallas de ordenador
speech	Sintetizadores para navegadores de voz utilizados por personas discapacitadas
tty	Dispositivos textuales limitados como teletipos y terminales de texto
tv	Televisores y dispositivos con resolución baja

MEDIOS DEFINIDOS CON REGLAS @media EN CSS

Las reglas @media son un tipo especial de regla CSS que permiten indicar de forma directa el medio o medios en los que se aplicarán los estilos incluidos en la regla.

- Para especificar el medio en el que se aplican los estilos, se incluye su nombre después de @media.
- Si los estilos se aplican a varios medios, se incluyen los nombres de todos los medios separados por comas.

```
@media print {  
    body { font-size: 10pt }  
}  
@media screen {  
    body { font-size: 13px }  
}  
@media screen, print {  
    body { line-height: 1.2 }  
}
```

MEDIOS DEFINIDOS CON REGLAS @import EN CSS

Cuando se utilizan reglas de tipo @import para enlazar archivos CSS externos, se puede especificar el medio en el que se aplican los estilos indicando el nombre del medio después de la URL del archivo CSS:

```
@import url("estilos_basicos.css") screen;  
@import url("estilos_impresora.css") print;
```

MEDIOS DEFINIDOS CON ETIQUETA <link>

Si se utiliza la etiqueta <link> para enlazar los archivos CSS externos, se puede utilizar el atributo media para indicar el medio o medios en los que se aplican los estilos de cada archivo:

```
<link rel="stylesheet" type="text/css" media="screen" href="basico.css" />  
<link rel="stylesheet" type="text/css" media="print, handheld" href="especial.css" />
```

SELECTORES

Los selectores definen sobre qué elementos se aplicará un conjunto de reglas CSS. La declaración indica "qué hay que hacer" y el selector indica "a quién hay que hacérselo". Por lo tanto, los selectores son imprescindibles para aplicar de forma correcta los estilos CSS en una página.

SELECTOR UNIVERSAL

Se utiliza para seleccionar todos los elementos de la página. El siguiente ejemplo elimina el margen y el relleno de todos los elementos HTML:

```
* {  
  margin: 0;  
  padding: 0;  
}
```

SELECTOR DE TIPO ETIQUETA

Selecciona todos los elementos de la página cuya etiqueta HTML coincide con el valor del selector. El siguiente ejemplo selecciona todos los párrafos de la página:

```
h1 {  
  color: red;  
}  
  
h2 {  
  color: blue;  
}  
  
p {  
  color: black;  
}  
  
h1 {  
  color: #8A8E27;  
  font-weight: normal;  
  font-family: Arial, Helvetica, sans-serif;  
}  
  
h2 {  
  color: #8A8E27;  
  font-weight: normal;  
  font-family: Arial, Helvetica, sans-serif;  
}  
  
h3 {  
  color: #8A8E27;  
  font-weight: normal;  
  font-family: Arial, Helvetica, sans-serif;  
}
```

SELECTOR DESCENDENTE

Selecciona los elementos que se encuentran dentro de otros elementos. Un elemento es descendiente de otro cuando se encuentra entre las etiquetas de apertura y de cierre del otro elemento.

```
p span { color: red; }
```

Si el código HTML de la página es el siguiente:

```
<p>
...
<span>texto1</span>
...
<a href="">...<span>texto2</span></a>
...
</p>
```

CSS O FLEX BOX

La diferencia básica entre CSS Grid Layout y CSS Flexbox Layout es que Flexbox se creó para diseños de una dimensión, en una fila o una columna.

- En cambio, CSS Grid Layout se pensó para el diseño bidimensional, en varias filas y columnas al mismo tiempo.
- Sin embargo, las dos especificaciones comparten algunas características comunes, y si ya has aprendido cómo utilizar Flexbox, verás semejanzas que te ayudarán a entender Grid.

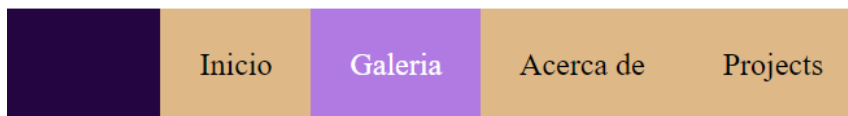
```
<!--Padre Contenedor-->
<div class="container-flex">
  <!--Hijos Contenedor-->
  <div style="flex-grow:3; order:1; background-color: red">1</div>
  <div style="order: 2; background-color: green">2</div>
  <div style="order: 3; background-color: hotpink">3</div>
  <div style="order: 4; background-color: indigo;color: white">4</div>
  <div style="order: 5; background-color: turquoise;">5</div>
  <div style="flex-grow:3; order: 6; background-color: yellowgreen;">6</div>
  <div style="order: 7; background-color: rgb(255, 95, 31);">7</div>
  <div style="order: 8; background-color: indigo;color: white">8</div>
</div>
```

Ejemplo de Navbar

```
<ul class="navigation">
  <li><a href="#">Inicio</a></li>
  <li><a href="#">Galeria</a></li>
  <li><a href="#">Acerca de</a></li>
  <li><a href="#">Projects</a></li>
</ul>
```

```
.navigation {  
  display: flex;  
  background-color: rgb(36, 5, 65);  
  list-style: none;  
  flex-flow: row wrap;  
  justify-content: flex-end;  
}  
  
a {  
  background-color: burlywood;  
  padding: 20px;  
  color: black;  
  text-decoration: none;  
  display: block;  
  transition: 0.9s;  
}
```

Resultado:



Como recurso para el aprendizaje de CSS FlexBox, se le entregara al estudiante varios ejercicios con interfaces del mundo real, donde se aplica Flexbox. Es importante que usted las pueda revisar para sus debidas prácticas.

Además, se facilitan los siguientes links, como recursos de aprendizaje:

Froggy CSS Flexbox: <https://flexboxfroggy.com/#es>

Curso de Flexbox: <https://www.youtube.com/watch?v=TtgkU8LMGAc&t=7s>



Tema 4: JAVASCRIPT

- JavaScript es el lenguaje de programación más popular del mundo.
- JavaScript es el lenguaje de programación de la Web.
- JavaScript es fácil de aprender.

¿Por qué estudiar JavaScript?

JavaScript es uno de los 3 lenguajes que todo desarrollador web debe aprender:

1. HTML para definir el contenido de las páginas web.

2. CSS para especificar el diseño de las páginas web.
3. JavaScript para programar el comportamiento de las páginas web.

Instrucciones JavaScript

Las instrucciones JavaScript se componen de:

- Valores, operadores, expresiones, palabras clave y comentarios.

```
let x, y, z;    // Statement 1
x = 5;         // Statement 2
y = 6;         // Statement 3
z = x + y;     // Statement 4
```

La mayoría de los programas JavaScript contienen muchas instrucciones JavaScript. Las declaraciones se ejecutan, una por una, en el mismo orden en que están escritas.

Programas JavaScript

- Un programa de computadora es una lista de "instrucciones" para ser "ejecutadas" por una computadora.
- En un lenguaje de programación, estas instrucciones de programación se denominan instrucciones.
- Un programa JavaScript es una lista de instrucciones de programación.

Punto y coma

Punto y coma separa las instrucciones javascript. Agregue un punto y coma al final de cada instrucción ejecutable:

```
let a, b, c;
a = 5;
b = 6;
c = a + b;
```

Cuando se separan por punto y coma, se permiten varias instrucciones en una línea:

```
a = 5; b = 6; c = a + b;
```

Espacio en blanco de JavaScript

JavaScript ignora varios espacios. Puede agregar espacios en blanco a su script para que sea más legible. Las siguientes líneas son equivalentes:

```
let person = "Hege";  
let person="Hege";
```

Una buena práctica es poner espacios alrededor de los operadores (= + - * /):

```
let x = y + z;
```

Palabras clave de JavaScript

Las instrucciones JavaScript a menudo comienzan con una palabra clave para identificar la acción de JavaScript que se realizará.

Palabra clave	Descripción
var	Declara una variable
let	Declara una variable de bloque
const	Declara una constante de bloque
if	Marca un bloque de instrucciones que se van a ejecutar en una condición
switch	Marca un bloque de sentencias a ejecutar en diferentes casos
for	Marca un bloque de instrucciones que se van a ejecutar en un bucle
function	Declara una función
return	Sale de una función
try	Implementa el control de errores en un bloque de instrucciones

Sintaxis de JavaScript

La sintaxis de JavaScript es el conjunto de reglas, cómo se construyen los programas de JavaScript:

1. Crear variables

```
var x;  
let y;
```

2. Usar variables


```
x = 5;  
y = 6;  
let z = x + y;
```

Literales de JavaScript

Las dos reglas de sintaxis más importantes para los valores fijos son:

1. Los números se escriben con o sin decimales:

```
10.50
```

```
1001
```

2. Las cadenas son texto, escrito entre comillas dobles o simples:

```
"John Doe"
```

```
'John Doe'
```

JavaScript Variables

En un lenguaje de programación, las variables se utilizan para almacenar valores de datos. JavaScript utiliza las palabras clave `var` y `let` para declarar variables.

- `var`
- `let`
- `const`

Se utiliza un signo igual para asignar valores a las variables. En este ejemplo, `x` se define como una variable. Entonces, a `x` se le asigna (dado) el valor 6:

```
let x;  
x = 6;
```

Operadores javascript

JavaScript utiliza operadores aritméticos (`+`) para calcular valores: `+-*/`

```
(5 + 6) * 10
```

Expresiones JavaScript

Una expresión es una combinación de valores, variables y operadores, que se calcula en un valor.

El cálculo se denomina evaluación.

Por ejemplo: $5 * 10$ evalúa a 50:

```
5 * 10
```

Las expresiones también pueden contener valores de variable:

```
x * 10
```

Los valores pueden ser de varios tipos, como números y cadenas.

Por ejemplo, `"John" + " " + "Doe"`, evalúa a `"John Doe"`:

```
"John" + " " + "Doe"
```

Comentarios en JavaScript

Los comentarios de JavaScript se pueden usar para explicar el código JavaScript y para hacerlo más legible. Los comentarios de JavaScript también se pueden usar para evitar la ejecución, al probar código alternativo.

Comentarios de una sola línea

Los comentarios de una sola línea comienzan con `//`

Cualquier texto entre `//` y el final de la línea será ignorado por JavaScript (no se ejecutará).

En este ejemplo se utiliza un comentario de una sola línea antes de cada línea de código:

```
let x = 5;      // Declare x, give it the value of 5
let y = x + 2;  // Declare y, give it the value of x + 2
```

Comentarios multilínea

Los comentarios de varias líneas comienzan con `/**` y terminan con `*/`

Cualquier texto entre `/**` y `*/` será ignorado por JavaScript

En este ejemplo se utiliza un comentario de varias líneas (un bloque de comentarios) para explicar el código:

```
/*  
Este es un comentarios de múltiples  
LINEAS  
*/
```

Funciones en JavaScript

Una función JavaScript es un bloque de código diseñado para realizar una tarea en particular. Una función JavaScript se ejecuta cuando "algo" la invoca (la llama).

```
<!DOCTYPE html>  
<html>  
<body>  
  
<p id="demo"></p>  
  
<script>  
function multiplicar(p1, p2) {  
    return p1 * p2;  
}  
document.getElementById("demo").innerHTML = multiplicar(4, 3);  
</script>  
  
</body>  
</html>
```

Sintaxis de la función JavaScript

Una función JavaScript se define con la palabra clave "function", seguida de un nombre, luego el paréntesis ().

Los nombres de las funciones pueden contener letras, dígitos, guiones bajos y signos de dólar (las mismas reglas que las variables).

Los paréntesis pueden incluir nombres de parámetros separados por comas:

(parámetro1, parámetro2, ...)

El código a ejecutar, por la función, se coloca entre corchetes rizados: {}

```
function name(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

- Los parámetros de función se enumeran dentro de los paréntesis () en la definición de función.
- Los argumentos de función son los valores recibidos por la función cuando se invoca.
- Dentro de la función, los argumentos (los parámetros) se comportan como variables locales.

Invocación de funciones

El código dentro de la función se ejecutará cuando "algo" invoque (llame) a la función:

- Cuando se produce un evento (cuando un usuario hace clic en un botón).
- Cuando se invoca (llama) desde código JavaScript.
- Automáticamente (autoinvocado).

Retorno de función

Cuando JavaScript alcanza una instrucción, la función dejará de ejecutarse con return.

Si la función se invocó desde una instrucción, JavaScript "volverá" a ejecutar el código después de la instrucción de invocación. Las funciones a menudo calculan un valor devuelto. El valor devuelto se "devuelve" al "llamador":

Ejemplo: Suma de dos números, usando función y retornando el resultado de la suma.

```
<!DOCTYPE html>  
<html>  
<body>  
  
<p id="demo"></p>  
  
<script>  
var x = suma(4, 3);  
document.getElementById("demo").innerHTML = x;  
  
function suma(a, b) {  
    return a + b;  
}  
</script>  
  
</body>  
</html>
```

Estructuras de control en JavaScript

La declaración if

Utilice la instrucción para especificar un bloque de código JavaScript que se ejecutará si una condición es true.

Sintaxis:

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

Ejemplo:

```
var edad = 12;  
if (edad >= 18) {  
    console.log("Usuario es mayor de edad " + edad);  
} else {  
    console.log("Usuario es menor de edad " + edad);  
}
```

Bucles en JavaScript

Do While

- Primero se colocan las operaciones y finalmente la condición.
- Debe tener un acumulador para que el algoritmo sea finito.

```
//Do while  
var i = 1;  
do {  
    console.log(i);  
    i += 1;  
} while (i <= 10);
```

Ejercicio que presenta los 10 primeros números enteros.

While

- Primero se coloca la condición y luego se procede a realizar las operaciones, dependiendo del estado de la condición.

```
var i = 1;
while (i <= 10) {
    console.log(i);
    i = i + 1;
}
```

For

El bucle tiene la sintaxis siguiente:

```
for (statement 1; statement 2; statement 3) {
    // code block to be executed
}
```

Ejemplo:

```
var i;
for (i = 1; i <= 10; i++) {
    console.log(i);
}
```

Arrays en JavaScript

Una matriz es una variable especial, que puede contener más de un valor:

```
const autos = ["Chevrolet", "Audi", "Mazda"];
```

¿Por qué usar matrices?

Si tiene una lista de elementos (una lista de nombres de automóviles, por ejemplo), almacenar los automóviles en variables individuales podría verse así:

```
let car1 = "Saab";
let car2 = "Volvo";
let car3 = "BMW";
```

Sin embargo, ¿qué pasa si desea recorrer los autos y encontrar uno específico? ¿Y si no tuvieras 3 coches, sino 300?

¡La solución es una matriz!

Una matriz puede contener muchos valores bajo un solo nombre y puede acceder a los valores haciendo referencia a un número de índice.

Creación de un arreglo de discos

Usar un literal de matriz es la forma más fácil de crear una matriz de JavaScript.

Sintaxis:

```
const array_name = [item1, item2, ...];
```

Ejemplo:

```
const cars = ["Saab", "Volvo", "BMW"];
```

Ejemplo # 2:

```
const cars = [  
  "Saab",  
  "Volvo",  
  "BMW"  
];
```

También puede crear una matriz y, a continuación, proporcionar los elementos:

```
const cars = [];  
cars[0]= "Saab";  
cars[1]= "Volvo";  
cars[2]= "BMW";
```

Uso de la palabra clave JavaScript new

En el ejemplo siguiente también se crea una matriz y se le asignan valores:

```
const cars = new Array("Saab", "Volvo", "BMW");
```

Acceso a los elementos del arreglo de discos

Para acceder a un elemento de matriz, consulte el número de índice:

```
const cars = ["Saab", "Volvo", "BMW"];  
let car = cars[0];
```

Cambio de un elemento de matriz

Esta instrucción cambia el valor del primer elemento en: cars

```
cars[0] = "Opel";
```

Ejemplo:

```
const cars = ["Saab", "Volvo", "BMW"];  
cars[0] = "Opel";
```

Objetos en JavaScript

Los objetos usan nombres para acceder a sus "miembros". En este ejemplo, devuelve
John = person.firstName

```
const person = {firstName:"John", lastName:"Doe", age:46};
```

Para un estudio profundo del lenguaje JavaScript, puedes revisar los siguientes tutoriales completos en: <https://www.w3schools.com/js/default.asp>

Para complementar esta unidad, dedicada especialmente al Frontend o Tecnologías en el lado cliente, usted deberá revisar la documentación oficial de los siguientes frameworks:



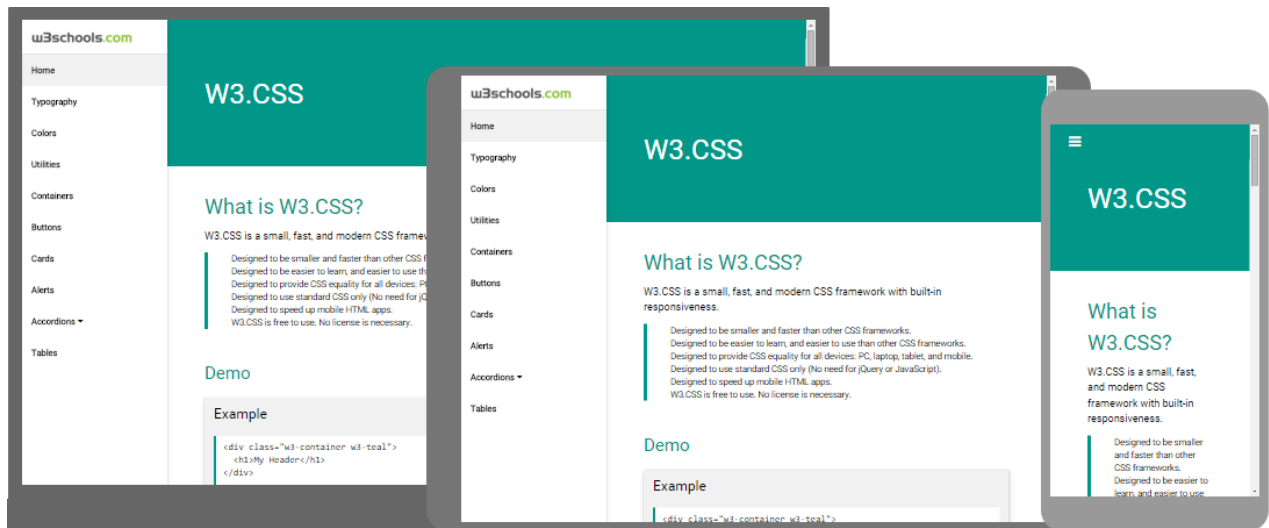
BOOTSTRAP



Link oficial: <https://getbootstrap.com/docs/4.6/getting-started/introduction/>



Curso complementario: <https://www.youtube.com/watch?v=nug1pMke-y4>



Link oficial: <https://www.w3schools.com/w3css/>



BULMA.io



Link oficial: <https://bulma.io/>



Curso complementario:

<https://www.youtube.com/watch?v=mjB7NrhC644&list=PLvKwKSrP7HoAbR5bXjwb4h5f2xjVxqdEe>

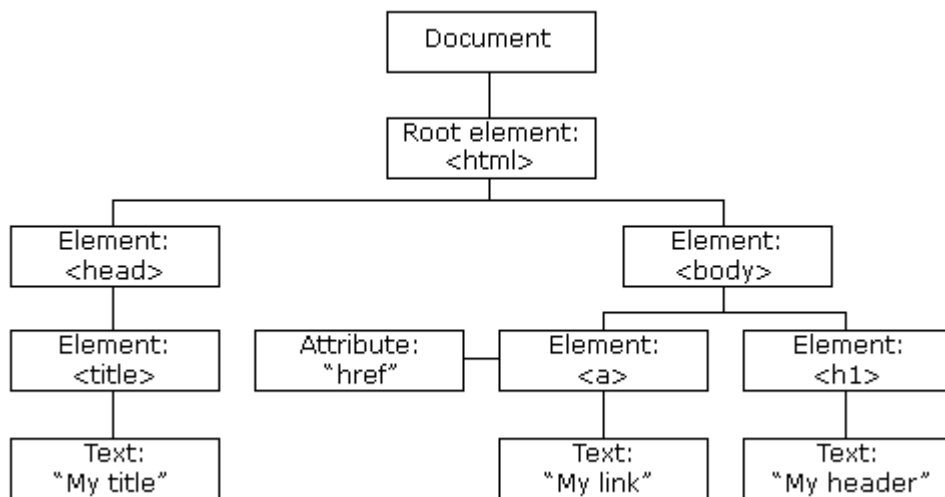


JAVASCRIPT HTML DOM

El DOM (Document Object Model)

El DOM es un estándar del W3C (World Wide Web Consortium). Con el DOM, JavaScript puede acceder y cambiar todos los elementos de un documento HTML.

Cuando se carga una página web, el navegador crea un Document Object Model de la página. El modelo DOM HTML se construye como un árbol de objetos:



Con el modelo de objetos, JavaScript obtiene toda la potencia que necesita para crear HTML dinámico:

- JavaScript puede cambiar todos los elementos HTML de la página.
- JavaScript puede cambiar todos los atributos HTML de la página.
- JavaScript puede cambiar todos los estilos CSS de la página.
- JavaScript puede eliminar elementos y atributos HTML existentes.
- JavaScript puede agregar nuevos elementos y atributos HTML.
- JavaScript puede reaccionar a todos los eventos HTML existentes en la página.
- JavaScript puede crear nuevos eventos HTML en la página.

Lo que aprenderás

En los siguientes capítulos de este tutorial aprenderás:

- Cómo cambiar el contenido de los elementos HTML.
- Cómo cambiar el estilo (CSS) de los elementos HTML.
- Cómo reaccionar a los eventos DOM HTML.
- Cómo agregar y eliminar elementos HTML.

¿Qué es el DOM HTML?

El DOM HTML es un modelo de objetos estándar y una interfaz de programación para HTML. Define:

- ◇ Los elementos HTML como objetos.
- ◇ Las propiedades de todos los elementos HTML.
- ◇ Los métodos para acceder a todos los elementos HTML.
- ◇ Los eventos para todos los elementos HTML.

En otras palabras: el DOM HTML es un estándar sobre cómo obtener, cambiar, agregar o eliminar elementos HTML.

JavaScript - HTML DOM Métodos

Los métodos DOM HTML son acciones que puede realizar (en elementos HTML).

La interfaz de programación DOM

- Se puede acceder al DOM HTML con JavaScript (y con otros lenguajes de programación).
- En el DOM, todos los elementos HTML se definen como objetos.
- La interfaz de programación son las propiedades y métodos de cada objeto.
- Una propiedad es un valor que puede obtener o establecer (como cambiar el contenido de un elemento HTML).
- Un método es una acción que puede realizar (como agregar o eliminar un elemento HTML).

Ejemplo:

En el ejemplo siguiente se cambia el contenido (el) del elemento con:

innerHTML a <p id="demo">

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dom # 1</title>
</head>

<body>
  <p id="demo"></p>
  <script>
    //escribe sobre la etiqueta p del HTML
    document.getElementById("demo").innerHTML = "Hola Mundo";
  </script>
</body>

</html>
```

El método getElementById

La forma más común de acceder a un elemento HTML es usar el del elemento.id

En el ejemplo anterior, el método utilizado para encontrar el elemento.getElementById → id="demo".

La propiedad innerHTML

La forma más fácil de obtener el contenido de un elemento es mediante la propiedad .innerHTML.

La propiedad es útil para obtener o reemplazar el contenido de los elementos HTML .innerHTML.

El objeto de documento DOM HTML

El objeto de documento representa la página Web.

Si desea acceder a cualquier elemento de una página HTML, siempre debe comenzar con el acceso al objeto de documento.

A continuación, se muestran algunos ejemplos de cómo puede utilizar el objeto de documento para acceder y manipular HTML.

Búsqueda de elementos HTML

Method	Description
<code>document.getElementById(<i>id</i>)</code>	Find an element by element id
<code>document.getElementsByTagName(<i>name</i>)</code>	Find elements by tag name
<code>document.getElementsByClassName(<i>name</i>)</code>	Find elements by class name

Cambio de elementos HTML

Property	Description
<code>element.innerHTML = <i>new html content</i></code>	Change the inner HTML of an element
<code>element.attribute = <i>new value</i></code>	Change the attribute value of an HTML element
<code>element.style.property = <i>new style</i></code>	Change the style of an HTML element
Method	Description
<code>element.setAttribute(<i>attribute</i>, <i>value</i>)</code>	Change the attribute value of an HTML element

Agregar y eliminar elementos

Method	Description
<code>document.createElement(<i>element</i>)</code>	Create an HTML element
<code>document.removeChild(<i>element</i>)</code>	Remove an HTML element
<code>document.appendChild(<i>element</i>)</code>	Add an HTML element
<code>document.replaceChild(<i>new</i>, <i>old</i>)</code>	Replace an HTML element
<code>document.write(<i>text</i>)</code>	Write into the HTML output stream

Búsqueda de elementos HTML

A menudo, con JavaScript, desea manipular elementos HTML.

Para hacerlo, primero debes encontrar los elementos. Hay varias maneras de hacer esto:

- Búsqueda de elementos HTML por id.
- Búsqueda de elementos HTML por nombre de etiqueta.
- Búsqueda de elementos HTML por nombre de clase.

- Búsqueda de elementos HTML mediante selectores CSS.
- Búsqueda de elementos HTML por colecciones de objetos HTML.

Búsqueda de elemento HTML por ID

La forma más fácil de encontrar un elemento HTML en el DOM es mediante el uso del identificador de elemento.

En este ejemplo se encuentra el elemento con :id="intro"

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dom # 3</title>
</head>

<body>
  <p id="intro"></p>
  <script>
    document.getElementById('intro').innerHTML = "Hola Mundo";
  </script>
</body>

</html>
```

Búsqueda de elementos HTML por nombre de etiqueta

En este ejemplo se encuentran todos los elementos:<p>

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dom # 4</title>
</head>

<body>
```

```
<p>1. </p><br>
<p>2. </p>
<p id="asignatura"></p>

<button type="button" onclick="cambio();">Aplicar cambio</button>

<script>
    function cambio() {
        var cambio = document.getElementsByTagName("p");
        var mensaje = document.getElementById("asignatura");
        mensaje.innerHTML = "DAW";
        cambio[0].innerHTML = "DAW DAW DAW";
    }
</script>

</body>

</html>
```

Búsqueda de elementos HTML por nombre de clase

Si desea encontrar todos los elementos HTML con el mismo nombre de clase, utilice `.getElementsByClassName()`

En este ejemplo se devuelve una lista de todos los elementos con `.class="intro"`.

```
<!DOCTYPE html>
<html lang="es">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dom # 5</title>
</head>

<body>
    <p class="len">PHP</p>
    <p class="len">JavaScript</p>
    <p class="len" style="background-color: yellow; font-family: 'Segoe
UI';">C++</p>
    <p class="len">C</p>
    <p class="len">C#</p>
    <p id="lenguaje"></p>

    <script>
        const l = document.getElementsByClassName('len');
```

```
l[2].innerHTML = "Go";  
</script>  
  
</body>  
  
</html>
```

Búsqueda de elementos HTML en formularios

```
<!DOCTYPE html>  
<html>  
  
<body>  
  
    <h2>JavaScript HTML DOM</h2>  
    <p>Encontrando elementos desde <b>Formulario</b>.</p>  
  
    <form id="frm1" action="">  
        First name: <input type="text" name="fname" value="Oscar"><br> Last name:  
<input type="text" name="lname" value="Lopez"><br><br>  
        <input type="submit" value="Envio">  
    </form>  
  
    <p>Estos son los datos que se han enviado desde el formulario</p>  
  
    <p id="demo"></p>  
  
    <script>  
        const x = document.forms["frm1"];  
        let datos = "";  
        for (let i = 0; i < x.length; i++) {  
            datos += x.elements[i].value + "<br>";  
        }  
        document.getElementById("demo").innerHTML = datos;  
    </script>  
  
</body>  
  
</html>
```

Cambiar el valor de un atributo HTML

Para cambiar el valor de un atributo HTML, utilice esta sintaxis:

```
document.getElementById(id).attribute = new value
```


En este ejemplo se cambia el valor del atributo src de un elemento:

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cambio de una imagen con DOM</title>
</head>

<body>
  

  <script>
    document.getElementById("mi_imagen").src =
"https://hotmart.com/media/2017/05/imagenes_para_anuncios.jpg";
  </script>

</body>
</html>
```

Contenido HTML dinámico (Fecha y Hora)

```
<!DOCTYPE html>
<html>

<body>
  <p id="demo"></p>

  <script>
    document.getElementById("demo").innerHTML = "Fecha y hora: " + Date();
  </script>

</body>
</html>
```

document.write()

En JavaScript, se puede utilizar para escribir directamente en el flujo de salida HTML:
document.write()

```
<!DOCTYPE html>
<html lang="es">
```

```
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dom # 9</title>
</head>

<body>
  <p>Ja ja ja ja ja ja</p>

  <script>
    document.write("Fecha y Hora: " + Date());
  </script>

  <p>Ja ja ja ja</p>

</body>
</html>
```

Validaciones con JavaScript

En este ejercicio se valida la entrada en el campo nombre del formulario.

```
<form name="formulario" action="" onsubmit="return validateForm()" method="post">
  Nombres: <input type="text" name="nombre">
  <input type="submit" value="Submit">
</form>
<script>
  function validateForm() {
    let x = document.forms["formulario"]["nombre"].value;
    if (x == "") {
      alert("El nombre debe ser rellenado");
      return false;
    }
  }
</script>
```

Si aprecian estimados estudiantes, se crea un evento onsubmit, lo que significa que el botón Submit, al ser presionado retornará lo que ejecuta la función validateForm().

La cual recibe el valor del elemento “nombre” del formulario y realiza una condición, si el campo está vacío, genera una alerta e invalida el envío.

Valida que un número pueda ser registrado cuando es igual o mayor que 1 e igual o menor que 10. En pocas palabras, debe estar comprendido de entre 1 a 10.

```
<!DOCTYPE html>
<html>

<body>

    <h2>Validación de Formulario</h2>

    <p>Ingrese un número del 1 al 10:</p>

    <input id="numb">

    <button type="button" onclick="verificar_num();">Registrar</button>

    <p id="demo"></p>

    <script>
        function verificar_num() {
            // Get the value of the input field with id="numb"
            let x = document.getElementById("numb").value;
            // If x is Not a Number or less than one or greater than 10
            let text;
            if (isNaN(x) || x < 1 || x > 10) {
                text = "Entrada no válida";
            } else {
                text = "Entrada correcta";
            }
            document.getElementById("demo").innerHTML = text;
        }
    </script>

</body>
```

Existen también validaciones directas en HTML, las cuales son:

Attribute	Description
disabled	Specifies that the input element should be disabled
max	Specifies the maximum value of an input element
min	Specifies the minimum value of an input element
pattern	Specifies the value pattern of an input element
required	Specifies that the input field requires an element
type	Specifies the type of an input element

Modificando con el DOM, elementos CSS

```
<!DOCTYPE html>
<html>

<body>

  <h2>JavaScript HTML DOM</h2>
  <p>Cambios de estilos CSS</p>

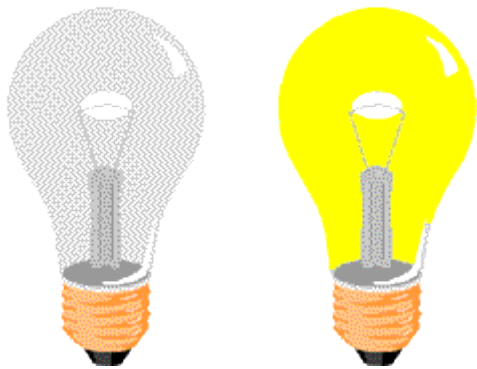
  <p id="p1">Hola Mundo</p>
  <p id="p2">Jugando con el DOM</p>

  <script>
    document.getElementById("p2").style.color = "blue";
    document.getElementById("p2").style.fontFamily = "Arial";
    document.getElementById("p2").style.fontSize = "larger";
  </script>

</body>

</html>
```

Ejercicio de estados del foco



```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Estado de Foco</title>
</head>

<body>
```

```
<button type="button" style="background-color: blue; color:white;"
onclick="prender();">Prender</button>
<button type="button" style="background-color: green; color:yellow;"
onclick="apagar();">Apagar</button>



<script>
    function prender() {
        document.getElementById('estado').src = "/imagenes/pic_bulbon.gif";
    }

    function apagar() {
        document.getElementById('estado').src = "/imagenes/pic_bulboff.gif";
    }
</script>
</body>
</html>
```

Referencias Bibliográficas

Bulma.io (2022). Framework Bulma CSS. Recuperado desde: <https://bulma.io/>

Peinado, F. (2019). Introducción al desarrollo web. Recuperado desde: <https://www.fdi.ucm.es/profesor/fpeinado/courses/webtech/Tema1-Introduccion.pdf>

W3School (2022). Curso de Bootstrap. Recuperado desde: <https://www.w3schools.com/bootstrap4/default.asp>

W3School (2022). Curso de CSS. Recuperado desde: <https://www.w3schools.com/css/default.asp>

W3School (2022). Curso de HTML. Recuperado desde: <https://www.w3schools.com/html/default.asp>

W3School (2022). Curso de W3 CSS. Recuperado desde: <https://www.w3schools.com/w3css/default.asp>